CEFET-MG | DEPARTAMENTO DE COMPUTAÇÃO

Computação Gráfica

Shaders

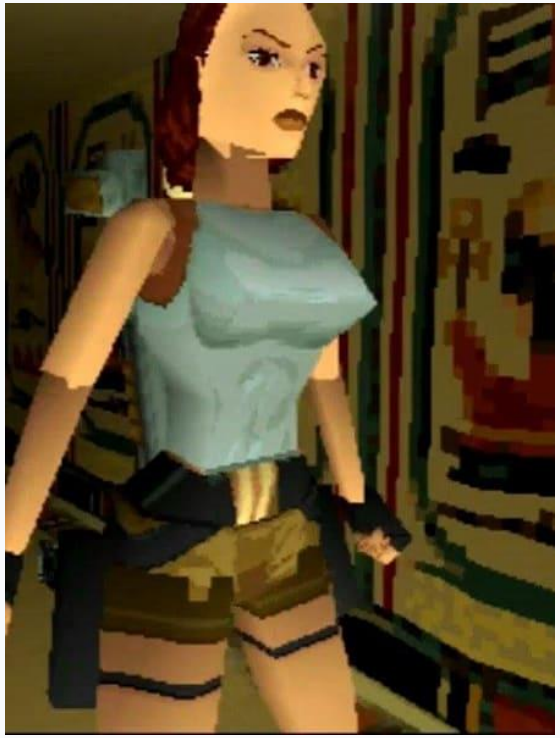
Prof. Dr. Raulivan Rodrigo

Contextualização

Vamos estudar agora é o que separa os sistemas antigos, como o PlayStation 1 dos gráficos mais realistas como o PlayStation 5.









Contextualização

- Até o final dos anos 90, o OpenGL possuía uma Pipeline de Função Fixa.
- O hardware tinha botões fixos: "Ligue a luz", "Aplique a neblina".
- Você não podia criar uma luz diferente, estava preso as funcionalidades placa de vídeo.



Contextualização

- Como solução, criou-se o Pipeline Programável.
- Removeram as "funções fixas“.

```
6   # SHADERS (A GPU pintando o azul)
7   VERTEX_SHADER = """
8   #version 330 core
9   layout (location = 0) in vec3 aPos;
10  layout (location = 1) in vec3 aColor;
11
12  out vec3 vertexColor;
13  uniform mat4 MVP;
14
15  void main() {
16      gl_Position = MVP * vec4(aPos, 1.0);
17      vertexColor = aColor; // Repassa o tom de azul para o fragmento
18  }
19  """
```

Shaders

Definição

Shaders são pequenos programas, escritos em uma linguagem própria muito parecida com C.

São compilados pela sua CPU.

São enviados para rodar exclusivamente dentro da sua Placa de Vídeo (GPU).

```
6   # SHADERS (A GPU pintando o azul)
7   VERTEX_SHADER = """
8   #version 330 core
9   layout (location = 0) in vec3 aPos;
10  layout (location = 1) in vec3 aColor;
11
12  out vec3 vertexColor;
13  uniform mat4 MVP;
14
15  void main() {
16      gl_Position = MVP * vec4(aPos, 1.0);
17      vertexColor = aColor; // Repassa o tom de azul para o fragmento
18  }
19  """
```

Shaders

O Paralelismo

- O processador (CPU) pode ter, por exemplo, 8 ou 16 núcleos.
- A placa de vídeo (GPU) possui milhares de núcleos menores.
- Se construirmos um modelo 3D com 1 milhão de vértices, a CPU levaria um tempo considerável calculando um por um, em um laço for.
- O Shader permite que a GPU calcule todos os 1 milhão de vértices ao mesmo tempo, no mesmo ciclo de clock.

Shaders

Tipos de Shaders

Existem vários tipos de Shaders, porem dois são mais usados:

01) Vertex Shader (Geometria):

- Executa uma vez para cada vértice.
- É aqui que fazemos as multiplicações matriciais (Translação, Rotação, Perspectiva) para posicionar os vértices no mundo.

Shaders

Tipos de Shaders

Existem vários tipos de Shaders, porem dois são mais usados:

02) Fragment Shader (Cores):

- Executa uma vez para cada pixel gerado na tela.
- É aqui que calculamos luz, sombras, texturas e as cores definitivas.

Shaders

Linguagem GLSL

Os Shaders são escritos em GLSL (*OpenGL Shading Language*).

A sintaxe é uma versão matemática da linguagem C.

```
21    FRAGMENT_SHADER = """
22    #version 330 core
23    in vec3 vertexColor;
24    out vec4 FragColor;
25
26    void main() {
27        FragColor = vec4(vertexColor, 1.0); // Pinta a tela
28    }
29    """
```

Shaders

Linguagem GLSL

As três palavras mais importantes da arquitetura GLSL são as rotas de comunicação:

- **in: Variável de Entrada**. Recebe dados da etapa anterior.
- **out: Variável de Saída.** Envia dados para a próxima etapa.
- **uniform: A Ponte Direta.** Uma variável global que a CPU (código Python) envia para a GPU. Ela se chama "uniforme" porque seu valor é o mesmo para todos os vértices e pixels durante aquele quadro específico (como a Matriz de Projeção).

Shaders

Na prática

- Para demonstrar a utilidade real de um Shader, vamos implementar um triângulo estático na memória.
- Depois, programar uma animação na GPU.
- Faremos ele pulsar e mudar de cor sozinho, movido pelo tempo.
- O código Python (CPU) apenas passará os "segundos" do relógio para o Shader.
- A GPU fará o resto.

Shaders

Na prática

- exemplo_shaders.py
- Código: https://github.com/raulivan/computacao-grafica-python/blob/main/AULA16/exemplo_shaders.py

exemplo_shaders.py U

AULA16 > exemplo_shaders.py > ...

```
1  import glfw
2  from OpenGL.GL import *
3  import OpenGL.GL.shaders
4  import numpy as np
5
```

```
# VERTEX SHADER
# Pega as posições brutas da memória Vertex Buffer Object (Objeto de Buffer de Vértices)
# ou simplismente (VBO).  e as converte para Coordenadas Homogêneas
VERTEX_SHADER_CODIGO = """
#version 330 core
layout (location = 0) in vec2 aPos; // Recebe X e Y do Python (VBO)

uniform float u_tempo; // Recebe o tempo do Python

void main() {
    // O vértice vai contrair ou inflar baseado na função seno do tempo!
    float pulsacao = sin(u_tempo) * 0.2;

    // gl_Position é uma variável nativa obrigatória (vec4: X, Y, Z, W)
    gl_Position = vec4(aPos.x + (aPos.x * pulsacao),
                       aPos.y + (aPos.y * pulsacao),
                       0.0, 1.0);
}
"""
```

Shc

Na prá

```
26  # FRAGMENT SHADER
27  # Calcula a cor de um pixel específico na tela
28  FRAGMENT_SHADER_CODIGO = """
29  #version 330 core
30  out vec4 CorDoPixel; // Saída final para o monitor
31
32  uniform float u_tempo; // recebe o mesmo tempo aqui
33
34  void main() {
35      // Usamos seno e cosseno para criar um RGB dinâmico
36      // Normaliza para ficar entre 0.0 e 1.0
37      float vermelho = (sin(u_tempo * 2.0) + 1.0) / 2.0;
38      float verde    = (cos(u_tempo * 1.5) + 1.0) / 2.0;
39      float azul     = (sin(u_tempo * 3.0) + 1.0) / 2.0;
40
41      // Pinta o pixel
42      // O último 1.0 é o Alpha (Opacidade)
43      CorDoPixel = vec4(vermelho, verde, azul, 1.0);
44  }
45  """
```

Shaders

```
47  def main():
48      print("Exemplo de usdo de Shaders...")
49
50      if not glfw.init(): return
51      glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
52      glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
53      glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFILE)
54
55      janela = glfw.create_window(800, 600, "Shaders na prática", None, None)
56      if not janela:
57          glfw.terminate()
58          return
59
60      glfw.make_context_current(janela)
61
```

Shaders

Na prática

```
61
62      # pedindo à GPU para compilar o código em C
63      programa_shader = OpenGL.GL.shaders.compileProgram(
64          OpenGL.GL.shaders.compileShader(VERTEX_SHADER_CODIGO, GL_VERTEX_SHADER),
65          OpenGL.GL.shaders.compileShader(FRAGMENT_SHADER_CODIGO, GL_FRAGMENT_SHADER)
66      )
67
```

```
    # definindo a geometria do triângulo SRO
    vertices = np.array([
        -0.5, -0.5,  # Vértice Esquerdo
         0.5, -0.5,  # Vértice Direito
         0.0,  0.5   # Vértice Topo
    ], dtype=np.float32)

    # Alocação Padrão de
    # VBO (Vertex Buffer Object)/ VAO (Vertex Array Object )
    VAO = glGenVertexArrays(1)
    VBO = glGenBuffers(1)

    glBindVertexArray(VAO)
    glBindBuffer(GL_ARRAY_BUFFER, VBO)
    glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)

    # definindo na GPU que cada vértice tem 2 floats (X e Y)
    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * vertices.itemsize, ctypes.c_void_p(0))
    glEnableVertexAttribArray(0)

    glBindVertexArray(0)
```

Shaders

Na prática: Importante destacar

- A placa de vídeo é extremamente rápida.
- Mas quando passamos o array para a GPU usando **glBufferData()**, a GPU vê apenas um oceano de bytes brutos (Zeros e Uns). Ela não sabe se aquilo são posições, cores, temperaturas ou coordenadas de textura.

```
glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)
```

Shaders

Na prática: Importante destacar

- De maneira simples:
- O VBO (*Vertex Buffer Object*) apenas transporta e despeja os bytes na memória da placa de vídeo.
- O VAO (*Vertex Array Object*) serve como tutorial de como a GPU deve ler aqueles bytes.

O método glBindVertexArray(VAO) diz para GPU que tudo que for configurado a partir de agora, guarde dentro deste de VAO.

Shaders

Na prática: Importante destacar

- **glVertexAttribPointer(0, ...)**
- Esse ZERO informado como parâmetro está relacionando ao location que declaramos lá no do shader
- **layout (location = 0)** in vec2 aPos;
- Cria um canal de comunicação.
- Ele vai pegar os valores enviado e informar na porta 0 (aPos) do Shader.
- Se no Shader estivesse escrito location = 5, no Python teríamos que escrever glVertexAttribPointer(5, ...)

Shaders

Na prática: Importante destacar

- **glEnableVertexAttribArray(0)**
- Diz: ligue a porta 0 para estabelecer a comunicação.

Shaders

Na prática: Importante destacar

- **ctypes.c_void_p(0))**
- O OpenGL é uma biblioteca feita em C, e a linguagem C lida com Ponteiros.
- O último parâmetro do **glVertexAttribPointer** informa quantos bytes é necessário pular desde o início da memória para começar a ler a posição.
- Como o array só tem coordenadas X e Y, os dados já começam imediatamente no byte zero. Por isso, o deslocamento é 0.
- Se o array fosse [X, Y, R, G, B], a posição começaria no byte 0, mas a cor começaria apenas após pularmos o X e o Y. Então, para a cor, o offset não seria zero, seria o tamanho em bytes de 2 floats.

Shaders

Na prática: Importante destacar

- **glBindVertexArray(0)**
- No OpenGL, o ID 0 é usado como a representação do estado Nulo / Desligado.
- Quando chamamos **glBindVertexArray(0)**, estamos fechando a comunicação e desvinculando o VAO.
- Voltando para o estado padrão (0).

```
# cria a Pasta (VAO) e o transporte (VBO)
VAO = glGenVertexArrays(1)
VBO = glGenBuffers(1)

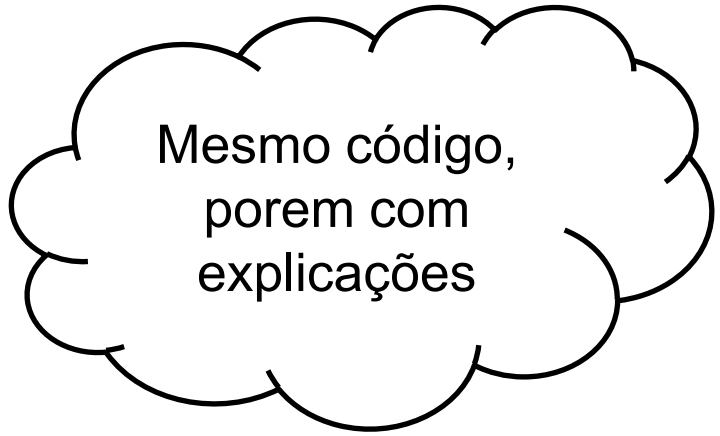
# abre a Pasta para começar a armazenar (Bind)
glBindVertexArray(VAO)

# transporta os Bytes na memória
glBindBuffer(GL_ARRAY_BUFFER, VBO)
glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)

# anota na Pasta
# Conecte os dados começando do byte 0 à Porta 0 do Shader.
# Leia de 2 em 2 floats.
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * vertices.itemsize, ctypes.c_void_p(0))

# anota na Pasta:
# Ligue a Porta 0
glEnableVertexAttribArray(0)

# terminei, fecha a pasta
glBindVertexArray(0)
```



Shaders

Na prática

```
90      # recuperando da GPU o endereço da variável 'u_tempo'"
91      local_tempo = glGetUniformLocation(programa_shader, "u_tempo")
92
93      print("calculando cores e tamanhos em tempo real...")
94
```

Shade

Na prática

```
95  while not glfw.window_should_close(janela):
96      glfw.poll_events()
97
98      # limpa o frame
99      glClearColor(0.1, 0.1, 0.1, 1.0)
100     glClear(GL_COLOR_BUFFER_BIT)
101
102     # usa o shader
103     glUseProgram(programa_shader)
104
105     # recupera o tempo do SO em segundos
106     tempo_atual = glfw.get_time()
107
108     # passando os parâmetros
109     # glUniform1f -> Passa um valor do tipo float
110     glUniform1f(local_tempo, tempo_atual)
111
112     # desenha
113     glBindVertexArray(VAO)
114     glDrawArrays(GL_TRIANGLES, 0, 3)
115
116     glfw.swap_buffers(janela)
117     # fim do loop
```

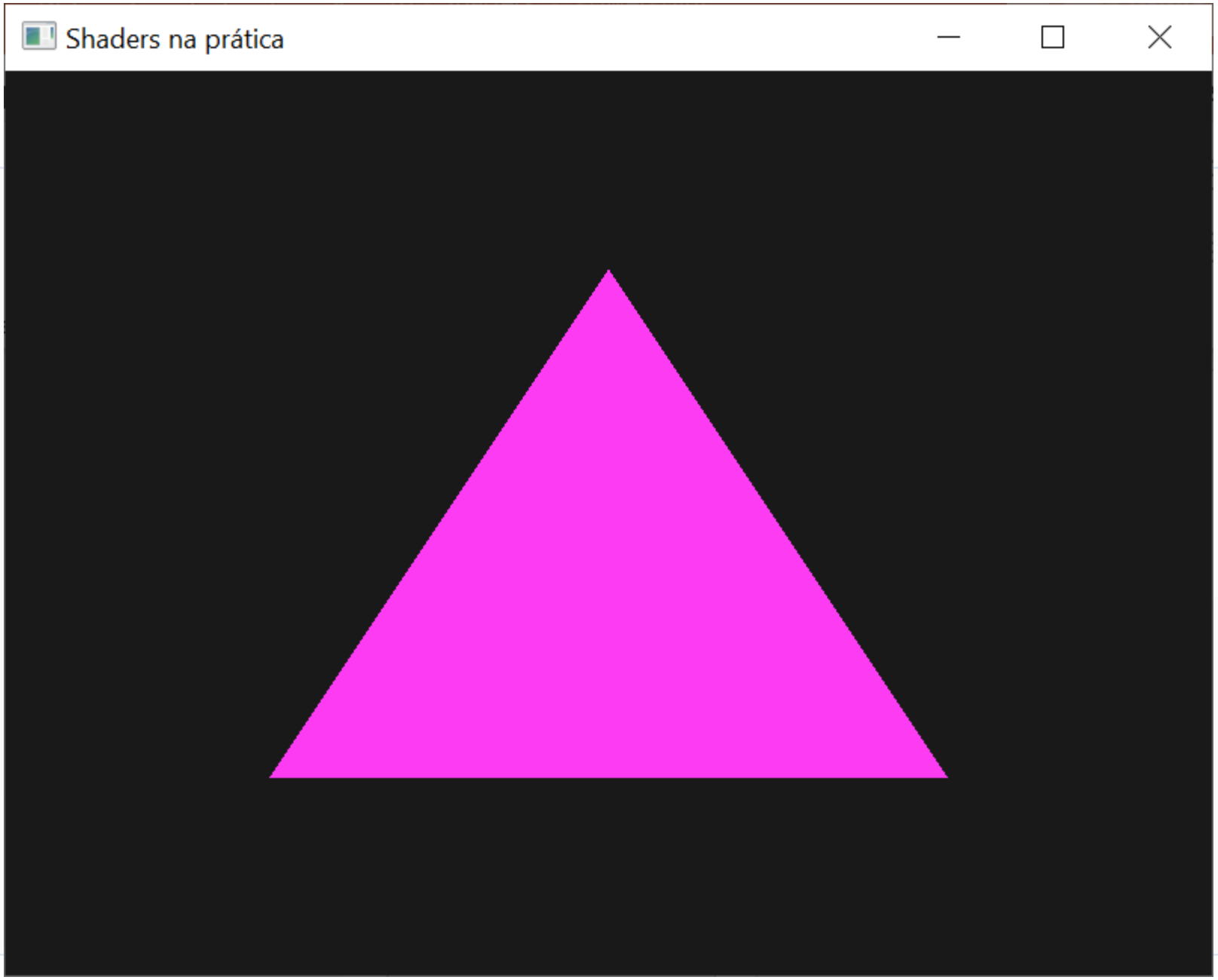
Shaders

Na prática

```
118
119         glfw.terminate()
120         # fim do método main
121
122     if __name__ == "__main__":
123         main()
```

Shaders

Na prática



Shaders

Na prática

Neste exemplo:

- O Python não está calculando o sin() para escalar as posições do triângulo.
- O Python não está iterando sobre os milhares de pixels dentro daquele triângulo para calcular o canal Vermelho, Verde e Azul de cada um deles.
- O Python está apenas recuperando o tempo (glfw.get_time()) e enviando para GPU.

A GPU recebe esse número pelo uniform float u_tempo.

Lá dentro, dispara simultaneamente o Fragment Shader para cada pixel da tela, aplicando a matemática das ondas seno/cosseno instantaneamente.

Shaders

Na prática

De maneira beeeeemmmmm simples:

- É assim que jogos renderizam oceanos inteiros se movendo ou gramas balançando com o vento
- Enviando o Tempo e a força do Vento via Uniforms para um Vertex Shader, permitindo que a Placa de Vídeo calcule o movimento ondulatório de 5 milhões de folhas de grama em frações de milissegundo.
- Enquanto a CPU fica livre para calcular a Inteligência Artificial dos NPCs.

Shaders

Mapeamento de Textura.

Até agora, os polígonos pareciam feitos de plástico liso.

Para resolver isso "embrulhamos" a geometria com imagens bidimensionais (como arquivos JPG ou PNG).

O nome dessa técnica é Mapeamento de Textura.





Shaders

Mapeamento de Textura: As Coordenadas UV

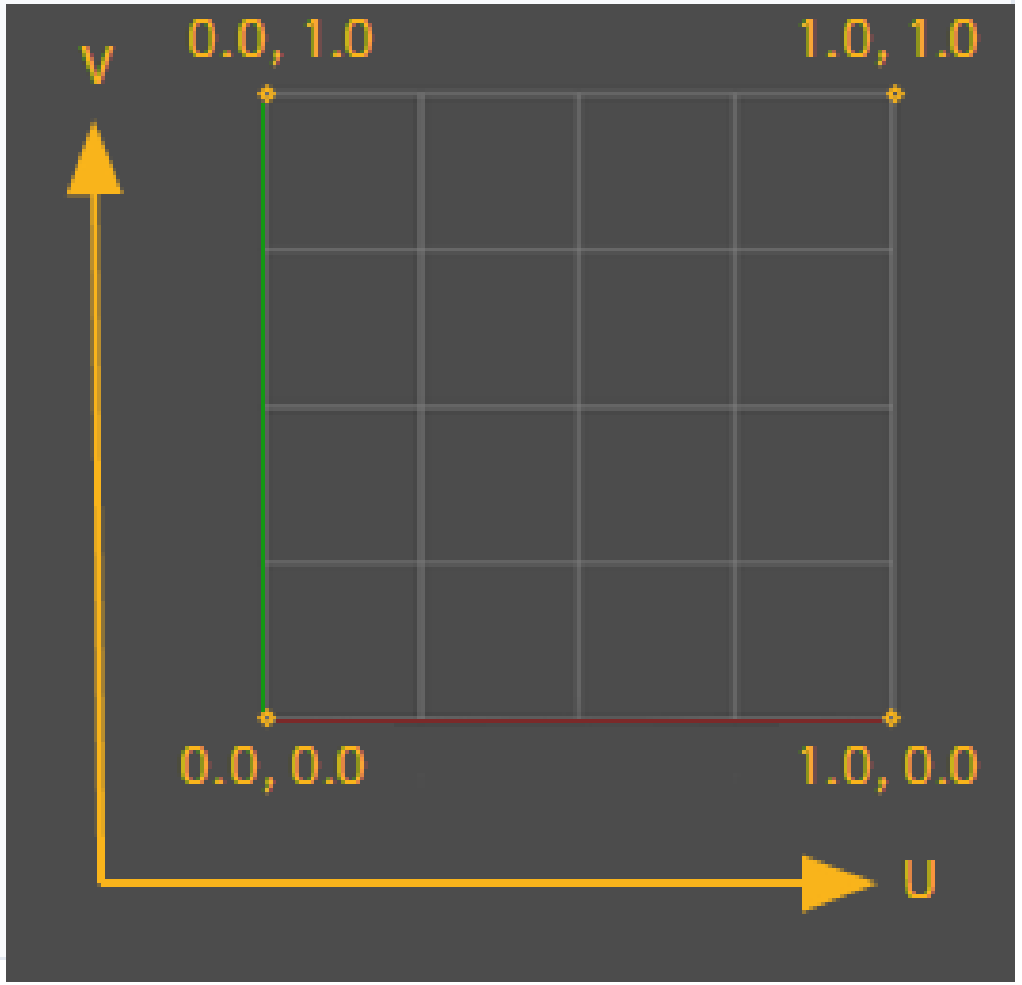
Como a GPU sabe qual parte da imagem (Textura) deve ser colada em qual vértice do nosso modelo 3D?

- É usado um sistema de coordenadas exclusivo para texturas, chamado de Plano UV (também conhecido como ST).
- O eixo U é a horizontal da imagem (vai de 0.0 a 1.0).
- O eixo V é a vertical da imagem (vai de 0.0 a 1.0).

Shaders

Mapeamento de Textura: As Coordenadas UV

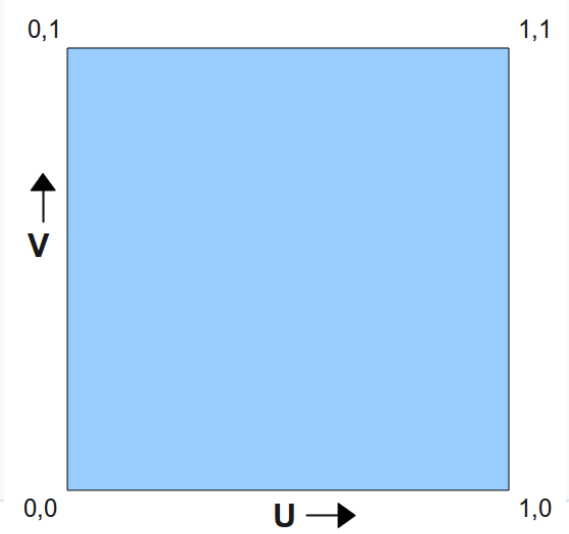
- A origem (0.0, 0.0) geralmente é o canto inferior esquerdo da imagem.
- O ponto (1.0, 1.0) é o canto superior direito.



Shaders

Mapeamento de Textura: As Coordenadas UV

- Se dissermos à GPU que o Vértice Superior Direito do seu quadrado 3D tem a coordenada UV (1.0, 1.0), o hardware saberá que deve colar o canto superior direito da foto exatamente ali.
- E, assim como nas cores, a GPU fará a interpolação automática para os pixels no meio do caminho.



Shaders

Mapeamento de Textura na prática

- exemplo_textura.py
- Código: https://github.com/raulivan/computacao-grafica-python/blob/main/AULA16/exemplo_textura.py

exemplo_textura.py U

AULA16 > exemplo_textura.py > ...

```
1   import glfw
2   from OpenGL.GL import *
3   import OpenGL.GL.shaders
4   import numpy as np
5
```

Shaders

```
6   # SHADERS preparados para textura
7   VERTEX_SHADER = """
8   #version 330 core
9   layout (location = 0) in vec3 aPos;     // PORTA 0: Recebe X, Y, Z
10  layout (location = 1) in vec2 aTexCord; // PORTA 1: Recebe U, V
11
12  out vec2 TexCord; // Variável para enviar o UV para o Fragment Shader
13
14  void main() {
15      gl_Position = vec4(aPos, 1.0);
16      TexCord = aTexCord; // Apenas repassa a coordenada UV
17  }
18  """
```

Shaders

Mapeamento de Textura na prática

```
20  FRAGMENT_SHADER = """
21  #version 330 core
22  in vec2 TexCord; // Recebe o UV interpolado da GPU
23  out vec4 FragColor;
24
25  // 'sampler2D' é um tipo especial do GLSL que acessa a memória de imagem da Placa de Vídeo
26  uniform sampler2D textura_xadrez;
27
28  void main() {
29      // A função texture() lê a cor exata do pixel da imagem naquela coordenada UV
30      FragColor = texture(textura_xadrez, TexCord);
31  }
32  """
```

Shaders

```
34  # Um método que gerar uma imagem de tabuleiro de xadrez
35  def criar_imagem_xadrez():
36      """Gera uma imagem de tabuleiro de xadrez 256x256 pixels em memória (RGB)"""
37      tamanho = 256
38      img = np.zeros((tamanho, tamanho, 3), dtype=np.uint8)
39      # criar o xadrez preto e branco
40      for i in range(tamanho):
41          for j in range(tamanho):
42              # o operador // é a divisão inteira.
43              # o operador % calcula o resto da divisão.
44              # verifica se a soma é par ou ímpar.
45              if (i // 32 + j // 32) % 2 == 0:
46                  img[i, j] = [255, 255, 255] # Branco
47              else:
48                  img[i, j] = [0, 0, 0]       # Preto
49      return img, tamanho, tamanho
50      # Fim do método
```

Shaders

```
53  def main():
54      if not glfw.init(): return
55      glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
56      glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
57      glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFILE)
58
59      janela = glfw.create_window(800, 600, "Mapeamento UV de Texturas", None, None)
60      if not janela:
61          glfw.terminate()
62          return
63
64      glfw.make_context_current(janela)
65
66      shader = OpenGL.GL.shaders.compileProgram(
67          OpenGL.GL.shaders.compileShader(VERTEX_SHADER, GL_VERTEX_SHADER),
68          OpenGL.GL.shaders.compileShader(FRAGMENT_SHADER, GL_FRAGMENT_SHADER)
69      )
70
```

Shaders

Mapeamento de Textura na prática

```
71      # QUADRADO SRO
72      # Formato: [X, Y, Z,     U, V]
73      vertices = np.array([
74           0.5,  0.5, 0.0,    1.0, 1.0, # 0: Topo Direito    (UV: Canto Superior Direito da Imagem)
75           0.5, -0.5, 0.0,    1.0, 0.0, # 1: Base Direita    (UV: Canto Inferior Direito)
76          -0.5, -0.5, 0.0,    0.0, 0.0, # 2: Base Esquerda   (UV: Canto Inferior Esquerdo)
77          -0.5,  0.5, 0.0,    0.0, 1.0  # 3: Topo Esquerdo   (UV: Canto Superior Esquerdo)
78      ], dtype=np.float32)
79
80      indices = np.array([0, 1, 3,  1, 2, 3], dtype=np.uint32)
81
```

Shaders

Mapeamento de Textura na prática

```
83      # glGenVertexArrays(1) -> O número 1 significa QUANTIDADE.
84      # a placa de vídeo, gere um ID numérico livre para eu usar como VAO
85      VAO = glGenVertexArrays(1)
86      VBO = glGenBuffers(1) # Gere 1 ID para VBO
87      EBO = glGenBuffers(1) # Gere 1 ID para EBO
88
89      glBindVertexArray(VAO)
90      glBindBuffer(GL_ARRAY_BUFFER, VBO)
91      glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)
92      glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO)
93      glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.nbytes, indices, GL_STATIC_DRAW)
94
```

Shaders

Mapeamento de Textura na prática

```
95      # cálculo do passo da memória:
96      # cada linha tem 5 floats (X,Y,Z, U,V). 5 * 4 bytes = 20 bytes.
97      passo = 5 * vertices.itemsize
98
99      # ATRIBUTO 0: POSIÇÃO GEOMÉTRICA (X, Y, Z)
100     # Parâmetro 1 (0): É a Porta do Vertex Shader (layout location = 0)
101     # Parâmetro 2 (3): lê 3 valores (X, Y, Z)
102     # Parâmetro 6 (0): Offset. Quantos bytes pular para começar a ler
103     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, passo, ctypes.c_void_p(0))
104     # ativa a Porta 0 do Shader
105     glEnableVertexAttribArray(0)
```

Shaders

Mapeamento de Textura na prática

```
107     # ATRIBUTO 1: COORDENADA DE TEXTURA (U, V)
108     # Parâmetro 1 (1): É a Porta do Vertex Shader (layout location = 1)
109     # Parâmetro 2 (2): lê 2 valores (U, V)
110     # Parâmetro 6 (3 * 4 bytes): Offset. Pule o X,Y,Z (3 floats) para achar o U e V.
111     glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, passo, ctypes.c_void_p(3 * vertices.itemsize))
112     # ativa a Porta 1 do Shader
113     glEnableVertexAttribArray(1)
114
115     # Desvincula o VAO passando 0 (0 = Nulo/Desligado)
116     glBindVertexArray(0)
117
```

Shaders

Mapeamento de Textura na prática

```
118     # definindo a textura
119     # imagem gerada no Python (Matriz RGB)
120     imagem_dados, largura, altura = criar_imagem_xadrez()
121
122     # glGenTextures(1) -> Gere um ID numérico livre para Textura
123     textura_id = glGenTextures(1)
124
125     # liga a Textura
126     glBindTexture(GL_TEXTURE_2D, textura_id)
127
```

Shaders

Mapeamento de Textura na prática

```
128     # configuração da textura
129     # Como a imagem se comporta se o UV passar de 1.0?
130     # GL_REPEAT faz ela se repetir (como papel de parede)
131     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
132     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
133     # Filtro: como a imagem se comporta ao darmos zoom?
134     # GL_NEAREST mantém o pixel quadriculado (estilo Minecraft)
135     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST)
136     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST)
137
```

Shaders

Mapeamento de Textura na prática

```
138     # envia os bytes da imagem (RAM) para a Placa de Vídeo (VRAM)
139     # o parâmetro '0' no meio: Level of Detail (LOD) ou Mipmap. 0 significa a imagem base principal.
140     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, largura, altura, 0, GL_RGB, GL_UNSIGNED_BYTE, imagem_dados)
141
142     # desvincula a textura (Passando 0)
143     glBindTexture(GL_TEXTURE_2D, 0)
144
145     print("quadrado com mapeamento UV.")
```

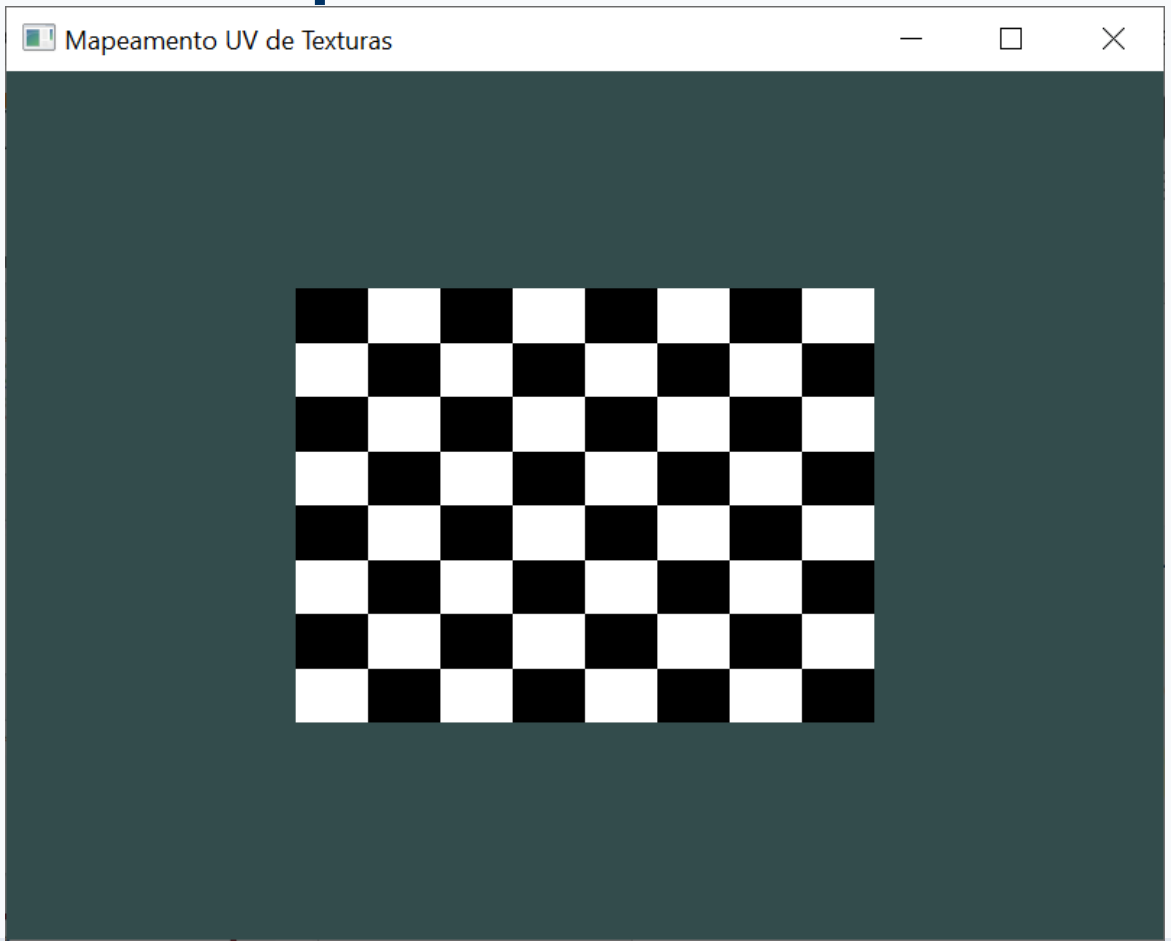
```
147     while not glfw.window_should_close(janela):
148         glfw.poll_events()
149         glClearColor(0.2, 0.3, 0.3, 1.0)
150         glClear(GL_COLOR_BUFFER_BIT)
151 
152         glUseProgram(shader)
153 
154         # a placa de vídeo tem várias portas físicas (Slots) para texturas simultâneas.
155         # GL_TEXTURE0 é o Slot Físico número 0.
156         # existe o GL_TEXTURE1, 2, etc até 31.
157         glActiveTexture(GL_TEXTURE0)
158         # Coloccolocaamos a nossa textura xadrez no Slot Físico 0
159         glBindTexture(GL_TEXTURE_2D, textura_id)
160 
161         # notificar o Shader: o sampler2D deve ler a imagem que está no Slot 0
162         loc_textura = glGetUniformLocation(shader, "textura_xadrez")
163         # "...1i" = envie 1 valor Inteiro (o número 0, referente ao Slot)
164         glUniform1i(loc_textura, 0)
165 
166         glBindVertexArray(VAO)
167         glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, None)
168 
169         glfw.swap_buffers(janela)
170         # fim do while
```

Shaders

```
172         glfw.terminate()
173         #Fim do main
174
175     if __name__ == "__main__":
176         main()
```

Shaders

Mapeamento de Textura na prática



Dúvidas e Discussão

Este é o momento para aprofundarmos nos conceitos.

raulivan@cefetmg.br

SIGAA

Próxima Aula

E vamos de Shaders...

